



ОБРАБОТКА ИСКЛЮЧЕНИЙ

Определение

Блок `EXCEPTION` – это специальная секция внутри блока `BEGIN ... END`, которая позволяет перехватывать и обрабатывать ошибки (исключения), возникающие при выполнении кода

Определение

vNum1 := 10;

vNum2 := 2;

vRes := vNum1 / vNum2;

Определение

```
DO $$
DECLARE
    vNum1 numeric;
    vNum2 numeric;
    vRes   numeric;
BEGIN
    vNum1 := 10;
    vNum2 := 2;
    vRes  := vNum1 / vNum2;
    RAISE NOTICE 'vRes = %', vRes;
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'There is error';
END;
$;
```

```
NOTICE:  vRes = 5.0000000000000000
DO
```

Определение

vNum1 := 10;

vNum2 := 0;

vRes := vNum1 / vNum2;



EXCEPTION

Определение

```
DO $$  
DECLARE  
    vNum1 numeric;  
    vNum2 numeric;  
    vRes  numeric;  
BEGIN  
    vNum1 := 10;  
    vNum2 := 0;  
    vRes  := vNum1 / vNum2;  
    RAISE NOTICE 'vRes = %', vRes;  
EXCEPTION  
    WHEN OTHERS THEN  
        RAISE NOTICE 'There is error';  
END;  
$$;
```

```
NOTICE:  There is error  
DO
```

Определение

BEGIN

vRes1 := vNum1 / vNum2;

EXCEPTION WHEN OTHERS THEN

RAISE NOTICE 'Error with vRes1';

END;

vRes2 := vNum1 * vNum2;

Определение

```
DO $$
DECLARE
    vNum1 numeric := 10;
    vRes1 numeric;
    vNum2 numeric := 0;
    vRes2 numeric;
BEGIN
    -- вложенный блок для vRes1
    BEGIN
        vRes1 := vNum1 / vNum2;
        RAISE NOTICE 'vRes1 = %', vRes1;
    EXCEPTION
        WHEN OTHERS THEN
            RAISE NOTICE 'Error with vRes1 ';
    END;

    -- внешний блок для vRes2
    vRes2 := vNum1 * vNum2;
    RAISE NOTICE 'vRes2 = %', vRes2;
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Error ';
END;
$$;
```

```
NOTICE:  Error with vRes1
NOTICE:  vRes2 = 0
DO
```

Функции в блоке исключений

SQLSTATE

возвращает код ошибки,
связанной с
исключительной
ситуацией

SQLERRM

возвращает сообщение
об ошибке, связанной с
исключительной
ситуацией

Функции в блоке исключений

```
WHEN OTHERS THEN  
  RAISE NOTICE '%',  
    'Error ' || SQLSTATE;
```

Функции в блоке исключений

```
DO $$
DECLARE
    vNum1 numeric := 10;
    vNum2 numeric := 0;
    vRes  numeric;
BEGIN
    vRes := vNum1 / vNum2;
    RAISE NOTICE 'vRes = %', vRes;
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE '%', 'Error ' || SQLSTATE;
END;
$$;
```

SQLSTATE

```
NOTICE:  Error 22012
DO
```

Функции в блоке исключений

SQLSTATE

Error Code	Condition Name
Class 00 — Successful Completion	
00000	successful_completion
Class 01 — Warning	
01000	warning
0100C	dynamic_result_sets_returned
01008	implicit_zero_bit_padding
01003	null_value_eliminated_in_set_function
01007	privilege_not_granted
01006	privilege_not_revoked
01004	string_data_right_truncation
01P01	deprecated_feature
Class 02 — No Data (this is also a warning class per the SQL standard)	
02000	no_data
02001	no_additional_dynamic_result_sets_returned

Функции в блоке исключений

SQLSTATE 22012

Класс 22 – Data Exception

22012 – division_by_zero

Деление на ноль.

Полный перечень кодов SQLSTATE и их расшифровку можно найти в официальной документации PostgreSQL (errcodes-appendix).

Функции в блоке исключений

**WHEN OTHERS THEN
RAISE NOTICE 'Error: %',
SQLERRM;**

Функции в блоке исключений

SQLERRM

```
DO $$
DECLARE
    vNum1 numeric := 10;
    vNum2 numeric := 0;
    vRes  numeric;
BEGIN
    vRes := vNum1 / vNum2;
    RAISE NOTICE 'vRes = %', vRes;
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Error: %', SQLERRM;
END;
$$;
```

```
NOTICE:  Error: division by zero
DO
```

Функции в блоке исключений

Исключение `OTHERS` перехватывает любую ошибку, но не дает понять, какая именно ошибка произошла.

- Чтобы обрабатывать разные ошибки по-разному, в PostgreSQL используются встроенные исключения.
- Каждому исключению соответствует свой код `SQLSTATE` и понятное имя, которое можно указать прямо в `WHEN`.

Встроенные исключения (1)

Условие (condition name)	SQLSTATE	Описание
division_by_zero	22012	Была сделана попытка деления на ноль.
case_not_found	20000	Не была выбрана ни одна из веток WHEN оператора CASE, а ветка ELSE отсутствует.
invalid_text_representation	22P02	Не удалось преобразовать текстовое значение в указанный тип данных (например, в число).

Встроенные исключения (2)

Условие (condition name)	SQLSTATE	Описание
no_data_found	P0002	Запрос SELECT ... INTO STRICT не вернул ни одной строки.
too_many_rows	P0003	Запрос SELECT ... INTO STRICT вернул больше одной строки.
unique_violation	23505	Была сделана попытка записать повторяющееся значение в столбец с ограничением уникальности.

no_data_found и too_many_rows срабатывают только у SELECT ... INTO STRICT. Обычный insert их не вызывает – для insert актуален unique_violation.

Встроенные исключения

```
vRes := vNum1 / vNum2;  
EXCEPTION  
  WHEN division_by_zero THEN
```

Встроенные исключения

```
DO $$
DECLARE
    vNum1 numeric := 10;
    vNum2 numeric := 0;
    vRes  numeric;
BEGIN
    vRes := vNum1 / vNum2;
    RAISE NOTICE 'vRes = %', vRes;
EXCEPTION
    WHEN division_by_zero THEN
        RAISE NOTICE 'Деление на ноль недопустимо';
    WHEN OTHERS THEN
        RAISE NOTICE 'Error: %', SQLERRM;
END;
$$;
```

DIVISION_BY_ ZERO

NOTICE: Деление на ноль недопустимо
DO

Встроенные исключения

```
CASE  
  WHEN vNum1 > vNum2 THEN ...  
  WHEN vNum2 > vNum1 THEN ...  
  -- ветки ELSE нет  
END CASE;
```

Встроенные исключения

```
DO $$
DECLARE
    vNum1 numeric := 10;
    vNum2 numeric := 10;
BEGIN
    CASE
        WHEN vNum1 > vNum2 THEN
            RAISE NOTICE 'Первое значение больше';
        WHEN vNum2 > vNum1 THEN
            RAISE NOTICE 'Второе значение больше';
    END CASE;
EXCEPTION WHEN case_not_found THEN
    RAISE NOTICE 'Условие не выполнено';
WHEN OTHERS THEN
    RAISE NOTICE 'Error: %', SQLERRM;
END;
$$;
```

CASE_NOT_FOUND

```
NOTICE: Условие не выполнено
DO
```

Встроенные исключения

```
SELECT ename, sal,  
       CASE  
         WHEN sal > 3000 THEN 'Высокий'  
         WHEN sal < 1000 THEN 'Низкий'  
         END AS sal_level  
FROM emp  
WHERE deptno = 10;
```

Встроенные исключения

```
SELECT ename,  
       sal,  
       CASE  
         WHEN sal > 3000 THEN 'Высокий'  
         WHEN sal < 1000 THEN 'Низкий'  
       END AS sal_level  
FROM emp  
WHERE deptno = 10;
```

CASE В SQL

ename	sal	sal_level
KING	5000.00	Высокий
CLARK	4999.00	Высокий
MILLER	1300.00	NULL

Ни одна ветка WHEN не подошла, ветки ELSE нет – запрос не падает, в sal_level возвращается NULL.

Встроенные исключения

```
vRes := CASE ... END; -- NULL  
CASE ... END CASE; -- ошибка
```

Встроенные исключения

```
DO $$
DECLARE
    vNum1 numeric := 10;
    vNum2 numeric := 10;
    vRes text;
BEGIN
    -- CASE-выражение: ошибки нет, результат NULL
    vRes := CASE
        WHEN vNum1 > vNum2 THEN 'Первое значение больше'
        WHEN vNum2 > vNum1 THEN 'Второе значение больше'
    END;
    RAISE NOTICE 'vRes = %', coalesce(vRes, 'NULL');

    -- CASE-оператор: ошибка case_not_found
    CASE
        WHEN vNum1 > vNum2 THEN
            RAISE NOTICE 'Первое значение больше';
        WHEN vNum2 > vNum1 THEN
            RAISE NOTICE 'Второе значение больше';
    END CASE;
EXCEPTION
    WHEN case_not_found THEN
        RAISE NOTICE 'Условие не выполнено';
END;
$$;
```

CASE В PL/pgSQL

```
NOTICE: vRes = NULL
NOTICE: Условие не выполнено
DO
```

Встроенные исключения

```
SELECT empno, job::numeric AS job  
FROM emp  
EXCEPTION  
WHEN invalid_text_representation  
THEN
```

Встроенные исключения

```
DO $$  
DECLARE  
    i RECORD;  
BEGIN  
    FOR i IN SELECT empno, job::numeric AS job FROM emp LOOP  
        RAISE NOTICE 'empno = %, job = %', i.empno, i.job;  
    END LOOP;  
EXCEPTION  
    WHEN invalid_text_representation THEN  
        RAISE NOTICE 'Job is not correct';  
    WHEN OTHERS THEN  
        RAISE NOTICE '%', 'Error ' || SQLERRM;  
END;  
$$;
```

INVALID_TEXT_
REPRESENTATION

NOTICE: Job is not correct
DO

Встроенные исключения

RAISE;

```
EXCEPTION  
WHEN division_by_zero THEN  
  RAISE NOTICE '...', SQLERRM;  
RAISE;
```

Встроенные исключения

```
DO $$
DECLARE
    vNum1 numeric := 10;
    vNum2 numeric := 0;
    vRes numeric;
BEGIN
    BEGIN
        vRes := vNum1 / vNum2;
    EXCEPTION
        WHEN division_by_zero THEN
            RAISE NOTICE 'Вложенный блок: %', SQLERRM;
            RAISE; -- передаём ошибку во внешний блок
    END;

    RAISE NOTICE 'Эта строка не выполнится';
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Внешний блок: % (код %)', SQLERRM, SQLSTATE;
END;
$$;
```

RAISE;

```
NOTICE: Вложенный блок: division by zero
NOTICE: Внешний блок: division by zero (код 22012)
DO
```

Собственные исключения

```
IF vBeginDate > vEndDate THEN  
    RAISE EXCEPTION  
        'Период указан некорректно'  
END IF;
```

Собственные исключения

```
DO $$
DECLARE
    vBeginDate date := to_date('01.06.2024', 'dd.mm.yyyy');
    vEndDate   date := to_date('01.05.2024', 'dd.mm.yyyy');
BEGIN
    IF vBeginDate > vEndDate THEN
        RAISE EXCEPTION 'Период указан некорректно';
    END IF;
EXCEPTION
    WHEN OTHERS THEN
        RAISE NOTICE 'Error: %', SQLERRM;
END;
$$;
```

```
NOTICE: Error: Период указан некорректно
DO
```

Собственные исключения

Собственное исключение создается оператором RAISE EXCEPTION с нужным текстом сообщения. При необходимости можно задать свой код ошибки и другие параметры через команду USING ERRCODE.

Собственные исключения

```
IF vNewSal > 6000 THEN  
  RAISE EXCEPTION 'Оклад %  
превышает лимит 6000', vNewSal  
  USING ERRCODE = 'P0002';  
END IF;
```

Собственные исключения

```
EXCEPTION  
WHEN SQLSTATE 'P0002' THEN  
RAISE NOTICE 'Лимит: % (код %)',  
SQLERRM, SQLSTATE;
```

Собственные исключения

```
DO $$
DECLARE
    vSal    numeric := 5000;
    vNewSal numeric;
BEGIN
    vNewSal := vSal * 1.5;

    IF vNewSal > 6000 THEN
        RAISE EXCEPTION 'Оклад % превышает лимит 6000', vNewSal
            USING ERRCODE = 'P0002';
    END IF;

    RAISE NOTICE 'Новый оклад: %', vNewSal;
EXCEPTION
    WHEN SQLSTATE 'P0002' THEN
        RAISE NOTICE 'Лимит: % (код %)', SQLERRM, SQLSTATE;
    WHEN division_by_zero THEN
        RAISE NOTICE 'Деление на ноль';
END;|
$$;
```

СВОЙ КОД ОШИБКИ

```
NOTICE: Лимит: Оклад 7500.0 превышает лимит 6000 (код U0001)
DO
```

Собственные исключения

Параметр USING	Описание
ERRCODE	Код ошибки SQLSTATE из 5 символов (цифры и заглавные латинские буквы). По умолчанию для RAISE EXCEPTION – P0001.
MESSAGE	Текст ошибки, если он не указан сразу после RAISE EXCEPTION.
DETAIL	Дополнительные подробности об ошибке.
HINT	Подсказка, как исправить ошибку.

Oracle и PostgreSQL

Oracle PL/SQL	PostgreSQL PL/pgSQL
<code>RAISE_APPLICATION_ERROR(-20001, 'Текст')</code>	<code>RAISE EXCEPTION 'Текст' USING ERRCODE = 'U0001'</code>
<code>PRAGMA EXCEPTION_INIT(eErr, -20001); WHEN eErr THEN</code>	<code>WHEN SQLSTATE 'U0001' THEN</code>
<code>SQLCODE / SQLERRM</code>	<code>SQLSTATE / SQLERRM</code>
<code>RAISE;</code>	<code>RAISE;</code>
<code>DBMS_UTILITY.FORMAT_ERROR_BACKTRACE</code>	<code>GET STACKED DIAGNOSTICS v = PG_EXCEPTION_CONTEXT</code>



**Обогащая традиции
инновациями**

Спасибо!
У вас остались вопросы?



* изображение для этого слайда создано нейросетью Kandisky 2.2 по запросу «Изображение в серых тонах, которое олицетворяет синергию традиций и инноваций в финансовой сфере»